BROWNFIELD · SPEC-DRIVEN · ORCHESTRATED

The Brownfield Run

Changing software an agent *didn't write* — safely, with a full audit trail.

~/asset-tracking-dockerize — zsh

```
# project:  Asset Decommission Approval  ·  Spring Boot 3.2.3 / Java 17 / PostgreSQL
# task:     turn an immediate soft-delete into a reviewed approval workflow
# method:   archaeology → characterize → narrow change → prove   (7 phase gates)
$ ./change.sh --reuse-not-replace --minimal-diff --prove-it
>>> a focused change, fully traceable, no regressions
```

FRAMING

Greenfield proved you can build.

Brownfield asks the harder question: **can an agent safely change what it didn't write?**

GREENFIELD

Build a new app from a spec pack.

The risk is capability — can we build it?
Nothing exists to break. The spec flows forward into code.

BROWNFIELD

Make one change to a working app.

The risk is hidden regressions — breaking what what already works.
You must understand and characterize first.

Same discipline. Inverted risk. The spec governs what should be true after — the code wins on what's true today.

OVERVIEW

What we'll cover

§1 The Brownfield Workflow

The fixed order, the five rules, and the audit audit layer that kept every change traceable. traceable.

§2 Archaeology

Phase 0 — knowing the repo before touching it. Surfacing surprises, not coding around them.

§3 The Change

One narrow, well-evidenced change that reuses the existing soft-delete instead of replacing it.

§4 Proof

Characterization before, regression after. Full requirement-to-evidence traceability.

§1

The Brownfield Workflow

Understand before you change. Characterize before you touch. Prove nothing else broke.

The fixed order — left to right, no skipping

You may loop back, but you never write business logic before the characterization tests are green.

CLAUDE.md §2 · the non-negotiable

"You may not modify business logic until the current behaviour is captured as a passing, repeatable check. Docker baseline → characterization → impact map → change → regression."

A change you cannot prove was safe is not done.

Five rules that made it safe

1	**Read before you write.**	docs/01, 02, 07 before any code; each phase re-read before it starts.
2	**Characterize before you change.**	Lock current behaviour as passing tests (C-01..05) before touching logic. logic.
3	**Reuse, don't replace.**	Approval triggers the existing @SQLDelete soft-delete — not a new delete path.
4	**Minimal diffs.**	Add new classes/templates. No renames, reformatting, or dependency bumps.
5	**Phase gates.**	Stop at each of 7 phases, summarize, wait for human approval before continuing.

The audit layer — a living wiki

A separate Dev-Wiki agent owns wiki/ and never touches the code it documents. Updated at every phase gate.

```
wiki/
├ WIKI_SCHEMA.md   how the wiki works
├ overview.md      phase tracker
├ glossary.md      soft-delete, pending…
├ log.md           every ingest + lint
├ features/        crud, request, approve
├ components/      controllers, service…
├ data-models/     asset, request
└ decisions/       ADR-001 … ADR-007

31 pages · 7 ADRs · per-phase log
```

Independent

A separate agent owns the wiki — it documents the code, never edits it.

Traceable

Every page cites the spec doc, requirement ID, or ticket it came it came from.

Verified

Each phase a code-drift lint greps the real source. Drift is surfaced, never overwritten.

Every change is traceable

Commit → Phase gate → Requirement / spec ID → ADR + wiki → Test / lint.

wiki/log.md · excerpt

```
## [2026-06-02] ingest | Phase 4 (data & service) — DecommissionService
   approve() reuses soft-delete (deleteById → @SQLDelete); N-01..N-07 green.
## [2026-06-02] lint  | Phase 4 code-drift check
   entity fields ⇔ DecommissionRequest.java — match. service API ⇔ code — match.
   generated DDL matches entity. Tests: 13 green. Links clean.   NO DRIFT.
```

Before the change — three numbers

What 'understand first' actually cost, and bought.

5

characterization tests green BEFORE any logic changed

0

structural changes to the Asset entity

2

ungated delete paths found — both closed

Next: **§2 · Archaeology** — knowing the repo before touching it.

§2

Archaeology

Phase 0 — produce a real map of the repo, verify the spec against the code, and surface what you can't answer.

Know it before you touch it

Phase 0 produced evidence from the real tree — no Docker, no business code yet.

`repo-map.md`	Every controller, endpoint, entity, repository, template — from the actual tree, not the spec.
`as-is-behaviour.md`	Current login / list / add / PATCH / delete behaviour, each verified and tied to a check.
`archaeology-questions.md`	The open questions the code can't answer — captured, not guessed.

Honesty on record: the original source repo was unreachable, so the baseline was reconstructed faithfully from docs/02 — recorded as ADR-007, flagged for reconciliation if the original appears.

Surface surprises — don't code around them

A finding the spec left open: how many ways can an asset actually be soft-deleted?

```
as-is finding · two delete paths
  GET  /assets-ui/delete/{id}   AssetUIController   ← the UI 'Delete' button uses this
  DELETE /assets/{id}           AssetController     ← REST path, NOT referenced by any UI/JS

@SQLDelete  → UPDATE assets SET is_deleted = true   (both paths trigger this)
@SQLRestriction('is_deleted = false')  → soft-deleted rows vanish from every read
```

The judgement call

Leaving a second, ungated path to soft-delete would let approval be bypassed. **Recorded as ADR-004** and seeded into the impact map — to be resolved in code, not quietly ignored.

§3

The Change

One narrow, well-evidenced change that reuses the existing soft-delete instead of replacing it.

The change in one sentence

Clicking **Delete** used to soft-delete instantly. Now it creates a **pending request** an approver must approve or reject — with a full audit trail.

BEFORE

Delete → immediate soft-delete
Asset vanishes from the dashboard.

No reason.
No approver.
No record of who or why.

AFTER

Request Decommission (+reason)
→ PENDING; asset stays active, badge shown
Approve → reuses soft-delete; APPROVED
Reject → asset stays active; REJECTED
Every decision audited: who · when · why · decider · outcome · comment.

Impact map — minimal diff, planned first

Prefer adding new classes over editing many. All logic in one new service.

NEW (6)

DecommissionRequest entity
DecommissionStatus enum
DecommissionRequestRepository
DecommissionService (all rules)
DecommissionUIController
decommissions.html (approver view)

EDITED (3)

assets.html Delete → Request + badge
AssetUIController pending ids; retire route
AssetController REST DELETE → 405

Untouched: Asset entity, PATCH edit, security, dependency versions.

Three decisions, recorded:

assetId as plain Long (ADR-002, history survives soft-delete) · pending derived from a PENDING request (ADR-003) · REST DELETE REST DELETE neutralized (ADR-004)

Reuse, don't replace — the one constraint

Approval ends up calling the existing soft-delete. No new UPDATE; no physical delete.

```
DecommissionService.approve(requestId, approver, comment)
  req.setStatus(APPROVED);  req.setDecidedBy(approver);  req.setDecidedAt(now);
  requestRepository.save(req);
  assetRepository.deleteById(req.getAssetId());   // ← reuses the existing mechanism

@SQLDelete("UPDATE assets SET is_deleted = true WHERE id = ?")   // entity-level, untouched
```

ADR-001 · the most important constraint

'Decommission' ***is*** the existing soft-delete — now gated behind approval. The characterization tests that locked that behaviour (C-01/C-02) keep covering it after the change.

Loose ends, named and closed

Brownfield judgement and environment surprises — handled without touching the deliverable.

```
# the second delete path  (ADR-004)
AssetController DELETE /assets/{id}  →  405 Method Not Allowed  (no soft-delete)
UI GET /assets-ui/delete/{id}        →  removed (404)
   ⇒ after the change, NO ungated path to soft-delete remains.

# environment surprises  (sandbox only — committed files unchanged)
TLS-intercepting proxy broke in-container Maven  →  trusted host CA in a tagged base image
Docker Engine 29 rejected docker-java API 1.32  →  pinned api.version=1.44 (tests only)
```

Surfaced, diagnosed, and fixed in the open — ***the deliverable Dockerfile and compose files were never compromised. compromised.***

§4

Proof

Characterization before, regression after, and every requirement mapped to evidence.

The safety net — tests as the contract

Characterization (C) passed BEFORE the change; new behaviour (N) after. Real Postgres, real soft-delete.

CHARACTERIZATION · before

C-01 soft-delete hides, not removes
C-02 @SQLRestriction filters reads
C-03 PATCH partial update intact
C-04 create + list
C-05 auth gate
→ green on UNCHANGED code first

NEW BEHAVIOUR · after

N-01..02 request / reason required
N-03..04 approve / reject
N-05..06 duplicate / soft-deleted guards
N-07 audit + history (FA-09)
N-08 dashboard pending badge
+ REST-405 · history-of-deleted

19 tests · 0 failures · 0 errors — Testcontainers PostgreSQL for true @SQLDelete fidelity (not H2). Characterization never went red.
never went red.

Live regression in Docker — T-01..T-09

Run against the committed `docker compose up --build`. All pass.

T-01	up --build → app+db start (db-healthy gate)	✓
T-02	login admin/admin123	✓
T-03	create asset → appears	✓
T-04	PATCH partial update intact	✓
T-05	request → PENDING, NOT soft-deleted	✓
T-06	reject → REJECTED, asset active	✓
T-07	approve → APPROVED, soft-deleted	✓
T-08	duplicate pending → blocked	✓
T-09	down -v → DB cleared	✓

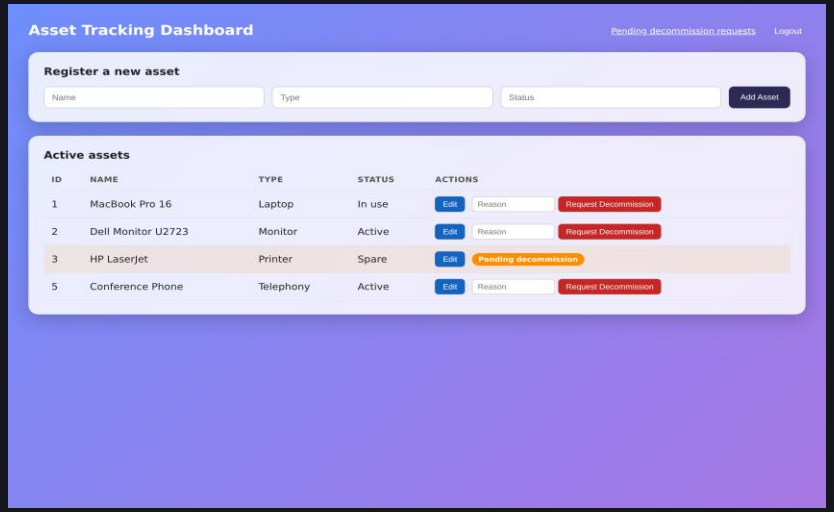
9 / 9

T-checks pass

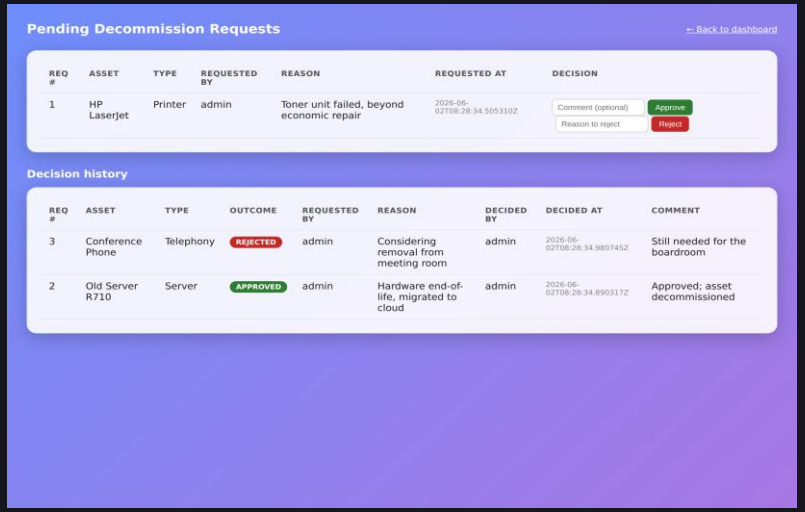
before/after demo recorded recorded

Evidence you can see — the running app

Captured from the live Dockerized app (headless Chromium).



Dashboard — pending badge + Request Decommission



Approver view — approve/reject + decision history (incl. soft-deleted)

Receipts

Locked at the final gate.

7
phase gates · all approved

19
tests · 0 fail / 0 error

9/9
success criteria (G-1..9) met

9/9
regression checks (T-01..09)

0
characterization regressions

1
new entity · Asset unchanged

0
ungated soft-delete paths left

7
artifacts + a 31-page wiki

Every requirement → evidence: **artifacts/requirements-traceability.md**

CLOSING

Same engine, adapted for brownfield

What carried over from the greenfield workflow — and what the brownfield risk demanded.

Spec pack precedes code	✓ carried over	Plus: verify the spec against the real code (it wins on 'today').
Living wiki + ADRs + lint	✓ carried over	Same independent agent; code-drift lint each phase gate.
Tests as the contract	adapted	Characterization FIRST — lock current behaviour before changing.
Phase gates / approval	adapted	7 gates; 'no logic before characterization is green'.
Smallest change wins	intensified	Reuse over rebuild; minimal diffs; one new service.
Traceability to evidence	✓ carried over	Requirement → code → test → live → doc, all mapped.

CLOSING

What this unlocks for brownfield work

Where the same approach pays off on code you already own.

Legacy modernization

Wrap an old service in Docker + characterization tests, then change it with a net.

Safe refactors

Lock behaviour as tests first; let drift checks catch anything anything you didn't intend.

Risky feature changes

Plan with an impact map; reuse existing mechanisms instead mechanisms instead of rebuilding.

Audit & compliance

A living wiki + requirement-to-evidence matrix makes 'prove it' a one-pager.

Stop hand-editing legacy code. Start directing a safe, evidenced change.

Spec on top.

Characterization underneath.

A narrow change in between.

```
$ git clone -b claude/jolly-albattani-ZVz7K  …/Asset-Tracking-Dockerize.git
$ mvn test                 # 19 green · Testcontainers Postgres
$ docker compose up --build # http://localhost:8080  (admin / admin123)
```

APPENDIX · artifacts/requirements-traceability.md

Requirement → evidence, every family ✓

The one page a reviewer reads to confirm done. All IDs mapped to code, test, live check, and doc.

G-1..G-9	Success criteria	live T-01..09 · N-tests · artifacts	✓
EB-01..06	Preserved behaviour	C-03/04/05 · live · untouched code	✓
FA-01..09	Functional reqs	N-01..08 · DecommissionService	✓
V-01..06	Validation rules	N-02/04/05/06 · guards	✓
UI-01..06	UI requirements	assets.html · decommissions.html · N-08	✓
DOCK-01..06	Dockerization	Dockerfile · compose · live T-01/T-09	✓